



UNIVERSITÀ DEGLI STUDI DI NAPOLI
FEDERICO II

DOCUMENTO DI DESIGN DEL SOFTWARE "BUGBOARD26"

CORSO DI INGEGNERIA DEL SOFTWARE

Gabriele Letizia - N86005267

Fabio Iannicelli - N86005383

ANNO ACCADEMICO 2025/26

Indice

1	Introduzione	3
1.1	Descrizione dello schema per la persistenza dati	4
1.2	Diagramma delle classi di design	6
1.3	Descrizione e motivazione delle scelte di software design	6
1.3.1	Architettura Multi-Tier e Pattern MVC	6
1.3.2	Data Transfer Object (DTO) Pattern	6
1.3.3	Repository Pattern	6
1.3.4	Singleton Pattern: Gestione della Sessione e IoC	7
1.3.5	Builder Pattern	7
1.3.6	Observer Pattern e Collezioni Osservabili	7
1.3.7	Architettura REST e Statelessness	7
1.3.8	Design della Sicurezza: Hashing delle Password	8
1.4	Codice Sorgente, Artefatti e Deployment	9
1.4.1	Build Automation (Maven)	9
1.4.2	Containerizzazione del Back-end (Dockerfile)	9
1.4.3	Orchestrazione e Rilascio in Cloud (Docker Compose e AWS EC2) . .	9

1 Introduzione

Il seguente documento descrive scelte architetturali, metodologie di design e artefatti software prodotti durante lo sviluppo di **BugBoard**, un'applicazione client-server dedicata alla gestione collaborativa di issue in progetti software.

Il progetto è stato concepito adottando un'architettura distribuita e cloud-ready, che separa nettamente la logica di presentazione (Front-end) dalla logica di business e persistenza dei dati (Back-end).

Il sistema si compone di un'interfaccia client desktop sviluppata in JavaFX e di un server RESTful basato su Spring Boot, interfacciato a un database relazionale PostgreSQL. Tramite Docker è stato possibile containerizzare l'infrastruttura di back-end, avendo così la possibilità di rilasciarlo in un ambiente di produzione su un'istanza cloud Amazon Web Services (AWS) EC2, garantendo isolamento, scalabilità e facilità di deployment.

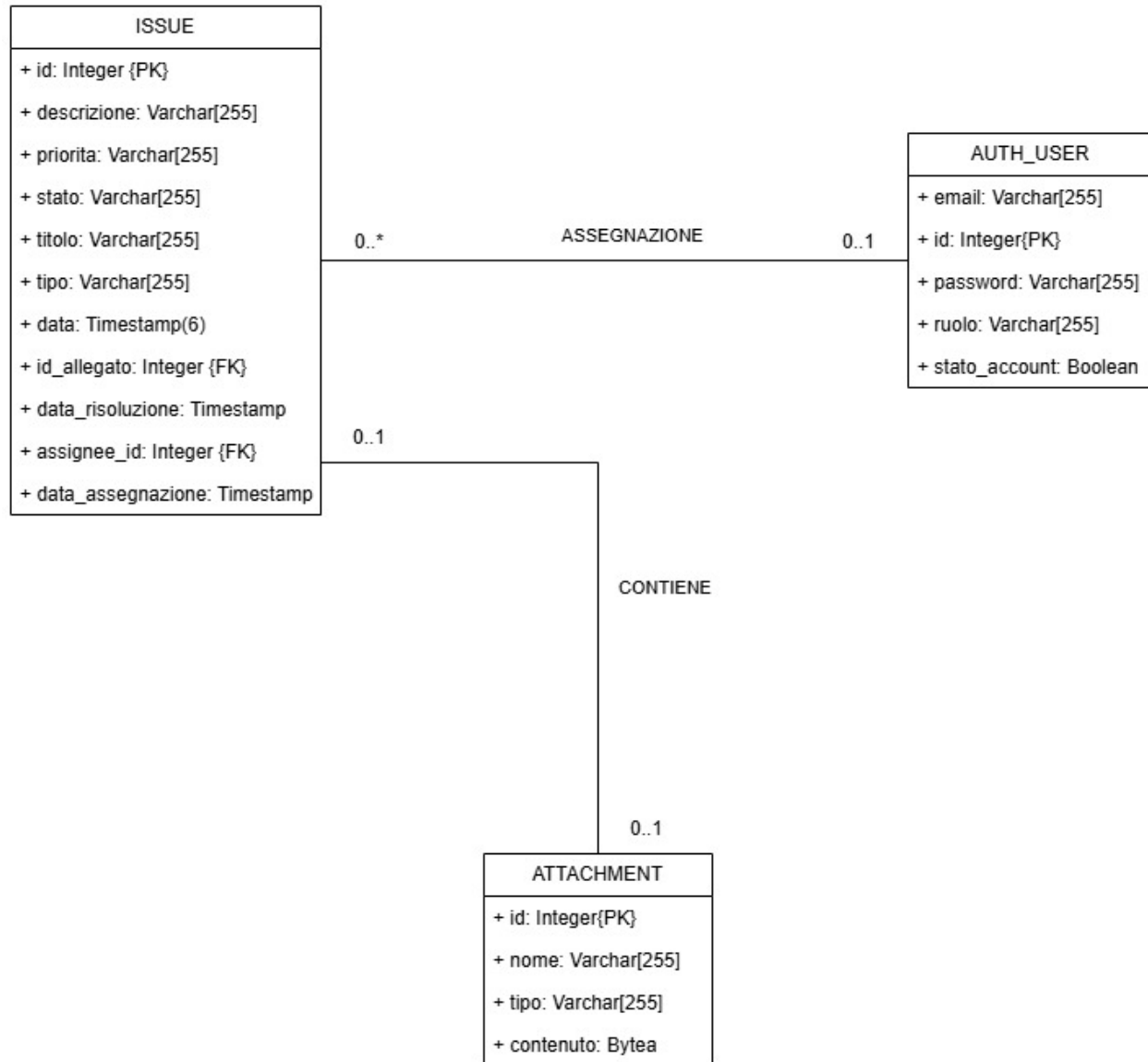
Al fine di garantire manutenibilità e qualità del codice, l'intero processo di sviluppo è stato supportato da pratiche moderne di Ingegneria del Software, tra cui il controllo di versione distribuito e l'analisi statica del codice.

Il documento è strutturato nelle seguenti sezioni:

- **Persistenza dei Dati:** Descrizione dello schema relazionale e delle strategie di memorizzazione adottate per il database PostgreSQL.
- **Design del Software:** Presentazione del diagramma delle classi di design, accompagnato dall'analisi e dalla motivazione dei design pattern e delle scelte architetturali adottate per risolvere specifici problemi di progettazione.
- **Processo di Sviluppo e Versioning:** Tracciamento delle attività del team tramite le metriche del repository GitHub.
- **Analisi della Qualità:** Esposizione dei report di ispezione del codice generati tramite strumenti di analisi statica (SonarQube) per il back-end, evidenziando la solidità e la sicurezza dell'implementazione.
- **Artefatti e Deployment:** Riferimenti al codice sorgente e alla configurazione degli strumenti di build automation (Maven) e containerizzazione (Dockerfile e docker-compose.yml) necessari per il rilascio del software.

1.1 Descrizione dello schema per la persistenza dati

Il livello di persistenza dei dati è stato progettato secondo il paradigma relazionale ed implementato su PostgreSQL. Lo schema fisico riflette in modo diretto il mapping Object-Relational delle entità di dominio definite nel backend (Spring Data JPA), garantendo consistenza tra la logica a oggetti e lo storage relazionale.



Le scelte architetturali e di design del database si articolano sui seguenti punti:

1. **Scelta delle Chiavi Primarie:** Per tutte le tabelle del sistema (auth_user, issue, attachment) si è optato per l'utilizzo di chiavi primarie surrogate (id di tipo Integer autoincrementante). Questa scelta disaccoppia l'identità fisica del record dalle informazioni di business, garantendo che eventuali modifiche anagrafiche non intacchino l'integrità referenziale delle chiavi esterne.

2. **Mappatura dei tipi di dato e degli Enumeratori:** I tipi di dato fisici sono stati scelti per ottimizzare lo storage e mantenere la leggibilità:

- **Enumeratori come Stringhe:** I campi che a livello applicativo sono gestiti tramite Enum (come ruolo, stato e tipo della Issue) sono stati mappati sul database come VARCHAR(255). Questa scelta garantisce una leggibilità immediata dei record eseguendo query dirette sul database e rende il sistema flessibile a futuri inserimenti di nuovi stati senza dover creare tipi custom complessi a livello SQL.
- **Persistenza dei File:** Per la gestione degli allegati si è scelto di salvare il file direttamente all'interno del database nella tabella attachment, utilizzando il tipo BYTEA. Considerando che il sistema prevede allegati leggeri, questa scelta centralizza il backup e semplifica il deployment, evitando la necessità di configurare un file system esterno o un bucket cloud.
- **Gestione Date:** Per la gestione delle date di transizione di stato delle issue (data, data_risoluzione, data_assegnazione) utilizzano il tipo TIMESTAMP. Questo permette tracciabilità esatta e calcolo preciso delle metriche operative.

3. **Implementazione delle Relazioni e Vincoli di Integrità referenziale:** Le associazioni tra le entità sono state modellate per rispettare le cardinalità del dominio, demandando al database il controllo sull'integrità referenziale:

- **Relazione ASSEGNAZIONE (Issue ↔ Auth_User | 1 a N):** Modellata tramite la Foreign Key assignee_id all'interno della tabella issue. Questo vincolo permette a una specifica issue di avere al massimo un solo utente assegnatario (0..1), mentre lo stesso utente può prendere in carico un numero non specificato di segnalazioni (0..*).
- **Relazione CONTIENE (Issue ↔ Attachment | 1:1):** Modellata tramite la Foreign Key id_allegato all'interno della tabella issue. Per imporre la cardinalità Uno-a-Uno ed evitare che uno stesso allegato venga associato per errore a più issue diverse, oltre al vincolo referenziale è stato applicato un vincolo di univocità (UNIQUE) sulla colonna id_allegato.

1.2 Diagramma delle classi di design

Il Diagramma delle Classi di Design illustra la struttura statica del sistema, evidenziando le classi principali, i loro attributi, i metodi e le relazioni che intercorrono tra di esse. Per gestire la complessità e favorire la separazione delle responsabilità, il sistema è stato logicamente suddiviso in due macro-componenti: il **Client** (Front-end JavaFX) e il **Server** (Back-end Spring Boot).

Visto l'elevato numero di classi, i due diagrammi non sono stati inclusi direttamente in questo documento, ma sono visualizzabili all'interno della cartella "**Class Diagram**", presente nella directory "**documentazione finale**" del materiale consegnato.

1.3 Descrizione e motivazione delle scelte di software design

La progettazione architetturale e di dettaglio del sistema è stata guidata dall'obiettivo di massimizzare la coesione, ridurre l'accoppiamento e favorire la manutenibilità del codice. Sono stati pertanto adottati diversi design pattern e stili architetturali standard.

1.3.1 Architettura Multi-Tier e Pattern MVC

L'intero sistema adotta un'architettura Client-Server distribuita. I due moduli sono organizzati secondo una separazione in livelli, con un'applicazione del pattern MVC particolarmente evidente nel Front-end JavaFX e una struttura a strati nel Back-end Spring Boot:

- **Front-end (Client):** Le interfacce grafiche (View) sono definite in modo dichiarativo tramite file FXML, separando nettamente il design dalla logica di interazione, che è demandata ai Controller JavaFX. La comunicazione di rete e le funzionalità di supporto sono state incapsulate in apposite classi di servizio e Helper, tra cui *ApiClient*.
- **Back-end (Server):** Segue una struttura a strati. Le richieste HTTP in ingresso vengono gestite dai *RestController* (Controller), che ne validano il formato e delegano la logica di business ai *Service*, i quali a loro volta interagiscono con il livello di persistenza tramite i *Repository*.

1.3.2 Data Transfer Object (DTO) Pattern

Per la comunicazione tra Front-end e Back-end sono stati utilizzati Data Transfer Object (DTO), evitando di esporre direttamente le entità JPA attraverso le API REST. I DTO contengono esclusivamente le informazioni necessarie alle singole operazioni, riducendo l'accoppiamento tra livello di persistenza e interfaccia applicativa e prevenendo problematiche legate alla serializzazione delle relazioni tra entità.

1.3.3 Repository Pattern

Per l'accesso ai dati persistenti è stato adottato il pattern **Repository**, implementato tramite il framework *Spring Data JPA*. I Repository forniscono un'astrazione sulle operazioni di persistenza, incluse le operazioni CRUD e le query necessarie all'applicazione, riducendo l'accoppiamento tra la logica applicativa e il livello di accesso ai dati. In questo modo, la gestione della persistenza risulta centralizzata e maggiormente manutenibile.

1.3.4 Singleton Pattern: Gestione della Sessione e IoC

Il pattern Singleton è stato sfruttato attivamente in entrambi i moduli del sistema, ma con due approcci differenti:

- **Lato Client (Manuale):** È stata implementata una classe *AuthSession* (o *UserSession*) come Singleton canonico. Questa scelta garantisce un unico punto di accesso globale per il salvataggio dello stato di autenticazione dell'utente (es. dati utente e token), rendendolo facilmente accessibile da qualsiasi controller grafico dell'applicazione senza dover passare l'oggetto di finestra in finestra.
- **Lato Server (Inversion of Control):** Spring gestisce il ciclo di vita dei componenti tramite il proprio container IoC. I bean, tra cui Controller, Service e Repository, utilizzano di default lo scope Singleton e vengono forniti alle classi che ne dipendono tramite Dependency Injection.

1.3.5 Builder Pattern

Per la configurazione del client HTTP nel Front-end, utilizzato per effettuare le chiamate REST verso il Back-end, deployato su infrastruttura AWS, è stato utilizzato il **Builder Pattern**. Sfruttando le API native di Java (*HttpClient.newBuilder()*), è stato possibile configurare in modo pulito e sequenziale parametri complessi, come i timeout di connessione, ottenendo infine un'istanza configurata e riutilizzabile del client HTTP.

1.3.6 Observer Pattern e Collezioni Osservabili

L'interfaccia utente in JavaFX sfrutta il meccanismo Observer attraverso le collezioni osservabili. Le liste utilizzate dai componenti grafici, come le tabelle delle issue, sono collegate ai dati sottostanti tramite *ObservableList*. Le modifiche apportate ai dati in memoria vengono automaticamente notificate ai componenti grafici interessati, che possono aggiornare la propria rappresentazione senza richiedere procedure manuali di refresh, garantendo un'interfaccia reattiva.

1.3.7 Architettura REST e Statelessness

Il livello di comunicazione tra client e server segue i principali vincoli dell'architettura REST, tra cui il principio di statelessness. Il Back-end non mantiene uno stato di sessione associato al singolo client tra una richiesta e l'altra.

Questa scelta consente al server di gestire ogni richiesta in maniera indipendente dalle richieste precedenti, mantenendo separato lo stato della sessione dal livello applicativo.

L'adozione di un'architettura stateless risulta inoltre coerente con il successivo deployment del Back-end su infrastruttura cloud AWS, poiché non richiede la persistenza di uno stato di sessione lato server.

1.3.8 Design della Sicurezza: Hashing delle Password

Per proteggere le credenziali degli utenti, il sistema adotta un approccio *Secure by Design*, evitando la memorizzazione delle password in chiaro all'interno del database. Per la memorizzazione sicura delle credenziali è stato scelto l'algoritmo di hashing **BCrypt**, la cui robustezza si basa principalmente su due meccanismi:

- **Generazione automatica del Salt:** BCrypt utilizza un valore casuale, denominato *salt*, per ogni password prima di calcolarne l'hash. In questo modo, due utenti con la stessa password produrranno hash differenti nel database, rendendo inefficaci gli attacchi basati su *rainbow tables*, ovvero tabelle pre-calcolate contenenti corrispondenze tra password e relativi hash.
- **Work Factor configurabile:** L'algoritmo è progettato per rendere computazionalmente oneroso il calcolo dell'hash. Il parametro *work factor* consente di configurare il costo computazionale dell'operazione, aumentando di conseguenza il tempo necessario per effettuare tentativi massivi di password e rendendo più onerosi gli attacchi di forza bruta.

1.4 Codice Sorgente, Artefatti e Deployment

Il processo di rilascio del software è stato strutturato al fine di favorire la riproducibilità degli ambienti di esecuzione e semplificare la generazione e la distribuzione degli artefatti. I codici sorgenti dei due moduli, Front-end e Back-end, sono accessibili tramite il repository GitHub citato in precedenza.

1.4.1 Build Automation (Maven)

Per entrambi i moduli, la gestione delle dipendenze e l'automazione del processo di build sono state affidate a **Apache Maven**. Tramite la configurazione dei rispettivi file *pom.xml*, Maven si occupa di:

- Risolvere e scaricare le librerie necessarie (come moduli JavaFX, driver PostgreSQL, Spring Boot Starter, BCrypt).
- Compilare il codice sorgente secondo la configurazione definita nel progetto.
- Generare gli artefatti di distribuzione, tra cui i file *.jar* prodotti nella cartella *target/*, destinati all'esecuzione o alla successiva containerizzazione del Back-end.

1.4.2 Containerizzazione del Back-end (Dockerfile)

Al fine di rendere l'ambiente di esecuzione del server maggiormente indipendente dalla configurazione del sistema operativo ospite e favorire la riproducibilità del deployment, l'applicativo Spring Boot è stato containerizzato.

A tale scopo è stato redatto un **Dockerfile** che definisce le istruzioni necessarie alla creazione dell'immagine dell'applicazione. Come immagine di base è stata selezionata ***eclipse-temurin:23-jdk-alpine***, che combina il JDK Eclipse Temurin con una distribuzione Linux minimale basata su Alpine. Tale scelta consente di contenere le dimensioni dell'immagine e di ridurre i componenti non necessari all'esecuzione dell'applicazione.

1.4.3 Orchestrazione e Rilascio in Cloud (Docker Compose e AWS EC2)

L'intera infrastruttura di back-end è gestita in modo dichiarativo tramite un file ***docker-compose.yml***. Questo strumento orchestra l'avvio simultaneo e la comunicazione di due servizi isolati:

- Il container dell'applicazione **Spring Boot**, costruito dinamicamente tramite il Dockerfile.
- Il container del database **PostgreSQL**, inizializzato con le configurazioni necessarie e dotato di volumi persistenti (*volumes*) per preservare i dati anche in caso di ricreazione o rimozione del container.

Grazie a questa configurazione, l'infrastruttura di Back-end è stata rilasciata in ambiente di produzione su un'istanza virtuale **Amazon Web Services (AWS) EC2** (classe *t3.micro*). Il client JavaFX, distribuito agli utenti finali come applicazione standalone, è stato configurato per comunicare tramite chiamate REST con l'indirizzo IP pubblico del server AWS, completando l'architettura distribuita del sistema.